

bpcGit

Consultant guide: setup, daily use and transports

This guide is for BPC consultants and administrators who set up and use bpcGit on SAP BPC 10.1 (NetWeaver). It explains how the app works, where things are stored, and what each screen does. For a non-technical introduction, see the separate *bpcGit overview*.

Screenshots show the real bpcGit screens with fictitious sample data (environment CORP_PLAN, models FINANCE and OPEX). Your environment, models and users will differ.

How it works

- **Runs inside SAP.** bpcGit is a UI5 application (BSP ZBPC_GIT) with an ABAP backend (HTTP handler ZCL_BPC_GIT_HTTP). It is installed with abapGit into package ZBPC_GIT.
- **Talks to Git through abapGit.** abapGit must be installed in the system. Git hosting can be GitHub or Bitbucket Cloud; Bitbucket adds faster history and diff reads and optional Git LFS for large workbooks.
- **Reads and writes BPC through BPC's own APIs:** the file service for documents, and the Data Manager, security, BPF and member APIs for the other objects.
- **One repository and branch per environment.** The setup is stored in table ZBPC_GIT_REPO; the last synced version of each object is stored in ZBPC_GIT_STATE.

Opening the app

```
/sap/bc/ui5_ui5/sap/zbpc_git/index.html?sap-client=<client>
```

Use this UI5 path rather than /sap/bc/bsp/sap/...: the BSP runtime rejects host names without a domain.

1. Repository setup

The Repository setup panel sits at the top of the page. When it is collapsed, its header still shows the saved repository and branch; **Change** opens the panel and puts the cursor in the Branch field.

bpcGit - Version control for BPC

Repository setup

bitbucket.org/acme-finance/bpc-content

Branch: main

Change

Choose an object type and optional model, then load its BPC and Git status.

The header shows the saved repository (without credentials) and branch.

Repository setup

bitbucket.org/acme-finance/bpc-content

Branch: main

Change

*BPC environment: CORP_PLAN

*Repository URL: https://bitbucket.org/acme-finance/bpc-content.git

BPC root folder: bpc

Changing this folder does not move existing Git files. Migrate them first; old sync baselines are cleared when the location changes.

Branch: main

Load branches

Large EPM workbooks: ☐ Use Git LFS (Bitbucket Cloud)

LFS threshold (MB): 5

1-100 MB

LFS applies to future commits of EPM workbooks at or above this size. Existing history is kept. LFS storage quota and SAP HTTPS access to the storage service are required.

Status: Last saved by ANNA.SMITH on 2026-10-01

Save

Connection

Git login: Not logged in. You are asked when the Git host needs it.

Read access: Not tested

Branch:

Push access:

Test connection

Log in and check push access

Repository setup for one BPC environment.

Field	What to enter
BPC environment	The environment this repository belongs to.
Repository URL	HTTPS URL of the Git repository. Credentials can be included as user : token@host (see below) or entered with Log in.
BPC root folder	Folder inside the repository that holds the BPC content, for example bpc. Leave empty to use the repository root.
Branch	Opening the list loads the branches; you can also type a new branch name.
Git LFS	Bitbucket Cloud only. EPM workbooks at or above the threshold (1-100 MB) are stored in LFS on their next commit.

Credentials

```
https://x-token-auth:YOUR_BITBUCKET_TOKEN@bitbucket.org/WORKSPACE/REPOSITORY.git
https://YOUR_GITHUB_USERNAME:YOUR_GITHUB_PAT@github.com/OWNER/REPOSITORY.git
```

- Bitbucket repository access token: user x - token - auth. Bitbucket API token: your Atlassian e-mail.
- GitHub fine-grained token: **Contents: Read-only** to read, **Read and write** to commit.
- A token in the URL is visible to every user who can open the environment's setup. **Log in** keeps credentials in the browser tab only.
- SAP must trust the Git host certificates (STRUST, with SNI). Bitbucket history and diff also need api.bitbucket.org.

Test connection checks read access and lists the branches; **Log in and check push access** confirms you can commit.

2. What is stored, and where

Every object becomes a file in the repository. Documents keep their BPC path below
\\ROOT\\WEBFOLDERS\\<ENV>\\ (with / instead of \\); objects stored in tables are written as generated XML.

Object	Repository path
EPM workbooks	<MODEL>/EEXCEL/REPORTS/...xlsx, .../INPUT SCHEDULES/..., also below <MODEL>/TEAM FILES/<TEAM>/
Logic scripts	ADMINAPP/<MODEL>/<NAME>.LGF (compiled .LGX files are not tracked)
Transformation files	<MODEL>/DATAMANAGER/TRANSFORMATIONFILES/...TDM plus its Excel workbook
Conversion files	<MODEL>/DATAMANAGER/CONVERSIONFILES/...CDM plus its Excel workbook
Data Manager packages	<MODEL>/DATAMANAGER/PACKAGES/<GROUP>/<PACKAGE>.xml
Package links	<MODEL>/DATAMANAGER/PACKAGELINKS/<NAME>.xml
Security	SECURITY/TEAMS/, SECURITY/TASKPROFILES/, SECURITY/DATAACCESSPROFILES/
BPF templates	<MODEL>/BPF/<technical name>.xml
Dimension members	DIMENSIONS/<DIM>/MEMBERS/<MEMBER>.xml

A transformation or conversion definition and its workbook are handled as a pair: they are committed, restored and transported together. BPC file names are upper case and kept as they are.

Excel workbooks are binary. Git keeps every version, but Diff does not show cell changes: for workbooks it reports only that the file changed.

3. The overview: what has changed?

Pick an **Object type** and optionally a **Model**, then **Load**. bpcGit compares BPC with the branch and lists each object with its status. The banner shows the branch, the commit compared against, and a count per status.

bpcGit - Version control for BPC

>

Repository setup

bitbucket.org/acme-finance/bpc-content

Branch: main

Change

Choose an object type and optional model, then load its BPC and Git status.

Object type

Logic scripts

Model

All models

Load

Files (9)

Commit (0)

Restore (0)

History

Diff

Add to transport (0)

Select BPC changes

Select Git changes

Search path

All models

...

Branch main at commit 4f1c2a9

Loaded in 0.4s

1 conflict, 1 modified in bpc, 1 modified in git, 1 new in bpc, 1 new in git, 4 unchanged

Status	File	Model	Location	Type	Last change in BPC	Size
☐ Unchanged	DEFAULT.LGF ADMINAPP	FINANCE	Admin	Logic script	2026-10-01 09:12 by ANNA.SMITH	2 KB
☐ Modified in BPC	CLEAR_FORECAST.LGF ADMINAPP	FINANCE	Admin	Logic script	2026-10-06 14:37 by J.TAYLOR	1 KB
☐ Unchanged	COPY_ACTUAL_TO_FCST. LGF ADMINAPP	FINANCE	Admin	Logic script	2026-10-02 11:05 by ANNA.SMITH	2 KB
☐ Modified in Git	CURRENCY_CONVERSION .LGF ADMINAPP	FINANCE	Admin	Logic script	2026-10-03 16:20 by M.CHEN	3 KB
☐ New in BPC	INTERCO_ELIMINATION.L GF ADMINAPP	FINANCE	Admin	Logic script	2026-10-06 10:48 by J.TAYLOR	3 KB
☐ Unchanged	DEFAULT.LGF ADMINAPP	OPEX	Admin	Logic script	2026-10-01 09:15 by ANNA.SMITH	1 KB
☐ Conflict	ALLOCATE_OVERHEAD.L GF ADMINAPP	OPEX	Admin	Logic script	2026-10-05 15:02 by M.CHEN	4 KB
☐ Unchanged	HEADCOUNT_COSTS.LGF ADMINAPP	OPEX	Admin	Logic script	2026-10-04 08:30 by R.PATEL	3 KB
☐ New in Git	SEED_BUDGET.LGF ADMINAPP	OPEX	Admin	Logic script		

Logic scripts of two models, with every common status.

Status	Meaning	Typical action
Unchanged	BPC matches the last synced Git version	None; History is available
Modified in BPC	Changed in BPC since the last sync	Diff, then Commit
New in BPC	Exists only in BPC	Commit
Modified in Git	Someone committed a newer version	Restore
New in Git	Exists only in Git	Restore
Conflict	Changed in BPC and in Git	Decide which side wins
Deleted in BPC / Git	Synced before, now missing on one side	Commit or Restore
Differs, never synced	Different, but never synced through bpcGit	Commit (BPC wins) or Restore (Git wins)

Select BPC changes and **Select Git changes** tick everything that can be committed or restored. The search box and the model, type and location filters help in large environments.

4. Diff: what exactly changed?

Select one logic script, transformation, conversion, package or package link and choose **Diff**. It compares the Git version with the current BPC version, line by line. Red lines with a minus are the Git version; green lines with a plus are what is in BPC now.

Diff - CLEAR_FORECAST.LGF

Git at 4f1c2a9 compared with current BPC. Minus lines are Git; plus lines are BPC.

ADMINAPP/FINANCE/CLEAR_FORECAST.LGF
Git: 228 bytes; BPC: 373 bytes. Different.

Line endings (CRLF/LF) are normalized for display. A final empty row indicates a trailing newline.

Git line	BPC line	Change	Content
1	1		// Clears forecast data before a new forecast cycle
	2	+	// Only open periods are cleared; closed periods keep their values
2	3		*XDIM_MEMBERSET CATEGORY = FORECAST
3	4		*XDIM_MEMBERSET TIME = %TIME_SET%
4		-	*XDIM_MEMBERSET AUDITTRAIL = INPUT
	5	+	*XDIM_MEMBERSET AUDITTRAIL = INPUT, MANUAL_ADJ
	6	+	*XDIM_FILTER TIME = [TIME].properties("PERIOD_STATUS") = "OPEN"
5	7		
6	8		*WHEN ACCOUNT
7	9		*IS *
8	10		*REC(EXPRESSION = 0)
9	11		*ENDWHEN
10	12		*COMMIT
11	13		

Close

CLEAR_FORECAST.LGF: a comment was added, the audit trail scope was widened and a filter for open periods was added in BPC.

5. History: who changed what, and when

Select an object and choose **History** to see every commit that changed it, newest first, with the author, date and commit message. Selecting a version and choosing **Restore selected version** puts that older version back into BPC, optionally recording it in a transport request.

History - COPY_ACTUAL_TO_FCST.LGF

All changes on this branch are shown.

☐

Clear forecast only for the selected time set
Anna Smith | 2026-10-02 11:05 | c41d9a07

Version

☒

Add MANUAL_ADJ audit trail to the clearing scope
Mei Chen | 2026-09-24 16:40 | 9b7e2c41

Version

☐

Initial version of forecast clearing
James Taylor | 2026-09-10 09:18 | 2e8f1a6c

Version

9b7e2c41f0a3d6e85c1b4a7f2e9d0c3b6a8f1e57
Add MANUAL_ADJ audit trail to the clearing scope

Transport request
No transport request

Restore selected version Close

History of a logic script with an older version selected.

6. Commit: save BPC changes to Git

Tick objects with a BPC-side change and choose **Commit**. The dialog lists what will happen to each object and asks for a message. All selected objects go into one commit, authored with your SAP user's name and e-mail.

The dialog box is titled "Commit 2 files". It contains the following information:

- Message: "One commit on branch main. The BPC version of each file goes to Git."
- File 1: "CLEAR_FORECAST.LGF" (FINANCE/ADMINAPP) with an "Update in Git" button.
- File 2: "INTERCO_ELIMINATION.LGF" (FINANCE/ADMINAPP) with an "Add to Git" button.
- Commit message field: "*Commit message" with the text "Clear forecast only for open periods; add intercompany elimination".
- Buttons: "Commit" and "Cancel".

Committing an updated and a new logic script together.

- Before writing, bpcGit checks that the branch has not moved since you loaded the list; if it has, reload and try again.
- Conflicts are never committed blindly: decide first whether BPC or Git wins.

7. Restore: put a Git version back into BPC

Tick objects and choose **Restore**. The dialog states what will be overwritten, created or deleted in BPC. A transport request can be chosen in the same dialog.

The dialog box is titled "Restore 1 file from Git" with a warning icon. It contains the following information:

- Message: "The Git version of branch main replaces the BPC version. The current BPC version is not kept by bpcGit unless it was committed before."
- File: "CURRENCY_CONVERSION.LGF" (FINANCE/ADMINAPP) with an "Update in BPC" button.
- Transport request dropdown: "No transport request".
- Buttons: "Restore" and "Cancel".

Restoring a newer Git version of a logic script.

bpcGit does not keep a copy of a BPC version that was never committed. Restoring over an uncommitted BPC change loses it. Files open for editing in BPC are skipped. Restoring dimension members processes the whole dimension, including other pending member edits.

8. Transport requests

bpcGit can put BPC objects into a **customizing transport request**, either with **Add to transport** (objects as they are in BPC now; nothing is restored) or while restoring. The request must be an open customizing request of the current client in which you have an open task; a new request can be created from the dialog on the default transport layer.

Adding two logic scripts to an open customizing request.

How objects are recorded

bpcGit records objects exactly as BPC's own transport does, so release and import work as usual:

- Each object is identified by a BPC *entity*: environment, model, entity type and entity ID. Its GUID is read from table UJT_GUID, or generated and stored there the first time. The same object always keeps the same GUID.
- The request gets an entry R3TR ABPC <GUID> (or the entity type itself for types with their own transport object) in your task.
- BPC exports the content when the request is **released**, so later changes to the object before release are included.

Object	Entity type	Entity ID
EPM workbook	AFLE	COMPANY\EEXCEL\... or <team>\EEXCEL\...
Transformation / conversion	ADMF	COMPANY\DATAMANAGER\<folder>\<name> (no extension)
Logic script	ASPR	ADMINAPP\<MODEL>\<NAME> . LGF
Data Manager package	ADMP	team, group and package ID
Package link	ADML	link name
Dimension members	AMBR	dimension (all members of the dimension)
BPF template	ABPF	template GUID
Team / task profile / data access profile	ATEM / ATPF / ADAF	ID

To see what a GUID stands for, look it up in UJT_GUID (SE16, field GUID); E071 shows which requests contain it. Deletions are not transported. An object that is already in the request or in one of its open tasks is not added twice; entries of released tasks are not counted.

9. Recommended ways of working

First baseline

After setup, load **All objects**, select the BPC changes and commit them with a message such as "Baseline of CORP_PLAN". From then on, every change shows up as a status.

A change from development to production

Step	In bpcGit
1. Change	Edit the script, report or package in the development system.
2. Review	Load the object type; check the change with Diff.
3. Commit	Commit with a message that explains why, and a ticket number if you have one.
4. Transport	Add to transport, then release the request as usual.
5. Recover	If something breaks, use History and Restore selected version.

Good habits

- Commit small, related changes together and write messages for the next consultant.
- Load and review statuses before a restore: Modified in BPC means someone has work that is not committed yet.
- Use branches for parallel work, for example a release branch per go-live.

10. Limits to know

- Targets SAP BPC 10.1 NW on ABAP 7.52 or later with UI5 1.52; abapGit must be installed.
- Workbook diffs show that a file changed, not which cells changed.
- Git LFS is supported for Bitbucket Cloud only (up to 128 MB per file).
- Deletions are not recorded in transport requests.
- Restoring a package over a custom script requires resetting the package to its default script in BPC first; scheduled package links cannot be deleted.